

Swift Robotics — Technical Approach & Interview Defense Guide

Perception Subsystem for Autonomous Mobile Robots (Door Open/Closed Detection)

Candidate: Saitejesh Tanuku | Junior AI Engineer Technical Evaluation

1. Executive Summary & Core Philosophy

Engineered as a production-grade robotics perception pipeline for AMRs. Highlights: Multi-source dataset normalization (\$2,512\$ images), 256-bit \$aHash\$ deduplication removing 369 burst frames (\$14.7\%\$), 6 controlled training experiments + 1 threshold sweep, real GPU/CPU latency profiling (PyTorch FP16, ONNX CUDA, ONNX CPU), and ROS2 / Nav2 safety integration.

2. Comprehensive Controlled Experiments & Validation Ablations

Exp #	Model	ImgSz	Key Factor Tested	Precision	Recall	F1	mAP50	mAP50-95	Latency (FPS)
Exp 1: Baseline	YOLOv8n	640	COCO Defaults Reference	0.9704	0.9690	0.9697	0.9757	0.8355	22.05 ms (45 FPS)
Exp 2: Augmentation	YOLOv8n	640	+HSV, Shear, Mixup	0.9696	0.9645	0.9670	0.9846	0.8197	21.23 ms (47 FPS)
Exp 3: High-Res	YOLOv8n	960	Spatial Scale (960px)	0.9791	0.9468	0.9627	0.9865	0.8327	26.56 ms (38 FPS)
Exp 4: Combined	YOLOv8n	800	Intermediate (800px)	0.9696	0.9673	0.9684	0.9844	0.8126	24.94 ms (40 FPS)
Exp 5: LR Schedule	YOLOv8n	640	LR=0.001 + AdamW	0.9680	0.9738	0.9709	0.9806	0.8462	17.73 ms (56 FPS)
Exp 6: Model Size	YOLOv8s	640	YOLOv8 Small (11.1M)	0.9800	0.9651	0.9725	0.9900	0.8455	18.80 ms (53 FPS)
Exp 7: Conf Sweep	YOLOv8n	640	Post-Hoc Conf Sweep	0.9739	0.9697	0.9718	0.9645	0.8259	22.05 ms (45 FPS)

3. Held-Out Test Evaluation & Robotics Safety Asymmetry

Ground Truth \ Predicted	Predicted door_open	Predicted door_closed	Background / Missed	Total Actual
Actual door_open (178)	172 (96.6%)	5 (2.8%)	1 (0.6%)	178 instances
Actual door_closed (103)	1 (1.0%) [Safety Risk]	97 (94.2%)	5 (4.8%)	103 instances

Robotics Safety Asymmetry: Safety-Critical False Traversability (Actual Closed → Predicted Open) occurred only **1 time out of 103 (<1.0%)**. Benign Fail-Safe Halts (Open → Closed) occurred 5 times (2.8%).

4. Hardware Latency Benchmarking (PyTorch CUDA vs ONNX CUDA vs ONNX CPU)

Runtime Engine	Execution Device	Precision	Mean Latency	P50 Latency	Throughput	Deployment Role
PyTorch Native	NVIDIA RTX 3050 GPU	FP16 (Half)	22.05 ms	18.40 ms	~45.3 FPS	Development Baseline
ONNXRuntime CUDA	NVIDIA RTX 3050 GPU	FP32 Static	25.52 ms	20.24 ms	~39.2 FPS	Exported Production Model
ONNXRuntime CPU	Host CPU (Default EP)	FP32 Static	73.66 ms	68.10 ms	~13.6 FPS	Cross-Platform Fallback

5. Key Interview Cross-Examination Defense

Q1: Why did you test Learning Rate schedules and Model Sizes?

To decouple optimization dynamics from capacity. Exp 5 proved that lowering initial LR to 0.001 with AdamW prevented weight shock, yielding the highest nano localization (mAP50-95: 0.8462). Exp 6 (YOLOv8s) proved extra capacity pushes precision to 98.0% and mAP50 to 99.0% while running at 53 FPS.

Q2: Why was dataset deduplication necessary?

CCTV streams contain 30 repetitive burst frames/sec. Pruning 369 duplicates (14.7%) via 256-bit aHash prevented train-test data leakage, ensuring test metrics represent true generalizability.

Q3: Why benchmark ONNX on both CPU and CUDA?

Isolates serialization overhead from execution acceleration. CPU ONNX ran at 73.66 ms, while CUDA ONNX achieved 25.52 ms (~39.2 FPS) on RTX 3050, matching native PyTorch FP16.

Q4: How does the model handle robotics safety asymmetry?

In mobile robotics, predicting closed as open is a lethal collision hazard, while predicting open as closed is fail-safe. Our model demonstrated a <1.0% false-traversability error rate on the held-out test set.

Q5: Why not trust single-frame YOLO output directly in Nav2?

Single-frame detectors suffer from transient sensor noise. A 3-frame temporal consensus filter eliminates false state flipping before updating costmaps.